B 2.3.1 Code Instructions Python

B 2.3.1 Construct programs that implement the correct sequence of code instructions to meet program objectives.Python

Program execution follows a precise order. This section explores these concepts with examples in Java and Python. It also explains how instruction sequencing impacts program behaviour and how to avoid common errors like infinite loops and incorrect output.

Learning Objectives

- Understand the importance of correct instruction order in programming.
- Identify and avoid common programming errors like infinite loops and deadlocks.
- Analyse code segments to predict output and identify potential issues.

Introduction

In computer programming, instructions are executed sequentially, one after the other. The order in which these instructions appear significantly impacts the program's functionality and output. This section explores the importance of correct instruction sequencing, common pitfalls, and strategies to avoid them.

Instruction Order and Program Functionality

Similar to Java, Python executes instructions sequentially. The interpreter reads and executes each line of code from top to bottom. The order in which instructions are written directly influences the program's behaviour and output.

Example 1

```
x = 5
y = 10
sum = x + y
print("The sum is:", sum)
```

This Python code mirrors the Java example. The variables x and y are assigned values, then their sum is calculated and stored in sum, and finally, the result is printed.

Example 2

```
print("The sum is:", 5 + 10)
```

Like in Java, Python evaluates the expression 5 + 10 before printing the result.

Common Errors and Avoidance Strategies

Infinite Loops:

Infinite loops in Python occur when a loop's condition remains true indefinitely

```
while True:
  print("This will print forever!")
```

Ensure your loop conditions can become false to prevent infinite loops.

Deadlocks:

Deadlocks in Python typically occur when multiple threads or processes are blocked, each waiting for the other to release a resource.

```
import threading

# Simplified example, actual deadlocks involve more complex scenarios
lock1 = threading.Lock()
lock2 = threading.Lock()

def thread1_function():
  with lock1:
    with lock2:
      # Access shared resources
      print("Thread 1 accessing resources")

def thread2_function():
  with lock2:
    with lock1:
      # Access shared resources
      print("Thread 2 accessing resources")

thread1 = threading.Thread(target=thread1_function)
thread2 = threading.Thread(target=thread2_function)

thread1.start()
thread2.start()
```

To avoid deadlocks, use consistent locking order and consider using techniques like timeouts or non-blocking locks.

Incorrect Output:

Incorrect output in Python can arise from logic errors, incorrect variable usage, or improper data type handling.

```
x = 5
y = "10"
print("The sum is:", x + y)
# TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

This code results in a TypeError because it attempts to add an integer (x) to a string (y). Ensure your variables have the correct data types for the operations you're performing.

Key Questions:

How does the order of instructions influence the execution of a Python program?

Python, like Java, is a sequential programming language. The interpreter executes instructions line by line, from top to bottom. The order you write your code directly determines the program's flow and the final outcome. If you try to use a variable before it's assigned a value or call a function before it's defined, Python will raise an error.

What are some common scenarios that lead to infinite loops in Python?

Infinite loops occur when the loop's condition never becomes false. Here are some common causes:

- **Incorrect loop condition:** Using an incorrect comparison operator (e.g., > instead of <) or a condition that's always true (e.g., while 1:).
- **Missing or incorrect loop variable updates:** Forgetting to increment a counter or modify a variable within the loop that's used in the loop condition.
- **Logic errors:** Mistakes in the code within the loop that prevent the condition from becoming false.

How can you prevent deadlocks when working with threads in Python?

Deadlocks happen when multiple threads are blocked, each waiting for another to release a resource. Here are some strategies to avoid them:

- **Consistent locking order:** Ensure all threads acquire locks in the same order to prevent circular dependencies.
- **Timeouts:** Use acquire(timeout=...) to acquire locks with a timeout. This allows a thread to give up after a certain time if the lock isn't available.
- **Try-finally blocks:** Use try-finally blocks to ensure that locks are always released, even if an exception occurs.
- **Context managers (with statement):** Use the with statement to acquire and release locks automatically, reducing the risk of forgetting to release them.

What are some debugging techniques for identifying and fixing incorrect output in Python programs?

Print statements: Use print() to display variable values at various points in your code to track the program's execution and identify unexpected values.
Debuggers: Python has built-in debuggers (like pdb), and IDEs provide debugging tools that allow

you to step through code, set breakpoints, and inspect variables.
Code review: Have someone else review your code for potential errors or logic issues.
Testing: Write unit tests using frameworks like unittest to test individual parts of your code and identify incorrect behaviour.
Error messages: Pay close attention to the error messages Python provides, as they often point to the location and type of error.